

第六次讨论课

排序技术研讨

经典算法设计技术研讨

中位数



1 概述

2 算法思想

3 求解过程

4 算法具体步骤

5 性能分析

中位数 概述

中位数问题的解决，用到中位数（Median）。

1、定义：一组数据按从小到大（或从大到小）的**顺序**依次排列，处在中间位置的一个数（或最中间两个数据的平均数）。
2、中位数和众数不同，众数指最多的数，众数有时不止一个，而中位数只能有一个；
3、中位数不一定在这组数据中。

无序数据	45 32 67 12 78 34
有序数据	12 32 34 45 67 78
中位数	(34+45)/ 2= 39.5

中位数 概述

问题算法思想：当个数为奇数时，取最中间位置的数；
当个数为偶数时，取最中间两个数的平均数。
假如有n个数据：

①当n为偶数时，中位数为第n/2位数和第 (n+2)/2位数的平均数；
②如果n为奇数，那么中位数为第（n+1）/2位数的值。

冒泡排序 算法思想

冒泡排序：
给定一组奇数数据，长度为n。

```
void bubsort(int A[], int n){
    for(int i=0 ; i<n-1 ;i++){
    for(int j=n-1 ; j>n-1 ;j--){
        if(A[j]>A[j+1]){
        int tem = A[j+1];
            A[j+1] =  A[j];
            A[j] = tem;        }}}
```



冒泡排序 性能分析

```
void bubsort(int A[], int n){
    for(int i=0 ; i<n-1 ;i++){
    for(int j=n-1 ; j>n-1 ;j--){
        if(A[j]>A[j+1]){
        int tem = A[j+1];
            A[j+1] =  A[j];
            A[j] = tem;       }}}
```

算法性能分析：在完全有序的情况下，最好的时间复杂度是O(n)，只需要1次冒泡。而在极端情况完全逆序，时间复杂度为O(n^2).

奇数序列排序之后，取（n+1）/2第四位的数字5即可。数组下标从0开始，所以是A[3]。

中位数切分

题目描述

给定一个长为n的数组a和一个整数m，你需要将其切成连续的若干段，使得每一段的中位数都大于等于m，求最多可以划分成多少段。

我们定义，偶数个数的中位数为中间两个数中较小的那一个，奇数个数的中位数就是正中间的数。如［2，3，1，5］的中位数为2，［2，3，1，2］的中位数为2，［3，5，9，7，11］的中位数为7。

示例1

输入 复制

```
4
5 4
10 3 2 3 2
5 3
5 2 3 3 2
2 5
4 5
5 2
10 3 2 3 2
```

输出 复制

```
-1
1
-1
5
```

CSDN @MangataTS

中位数切分

输入描述：

输入第一行是一个整数T（1≤T≤20），**测试组数T**。

每个测试第一行是**两个整数n，m**（1≤n≤105，1≤m≤109），含义如题目所示。第二行输入n个数，数组a，满足1≤ai≤109。

由于本题输入量较大，建议使用scanf等高效输入方式。

输出描述：

每个测试用例，输出一个整数，表示最多可以划分成多少段，若无论如何划分都不能满足条件，

示例1

输入 复制

```
4
5 4
10 3 2 3 2
5 3
5 2 3 3 2
2 5
4 5
5 2
10 3 2 3 2
```

输出 复制

```
-1
1
-1
5
```

CSDN @MangataTS

中位数切分 算法思想

解法

算法思想：排序完后，每次找到中间位置，看是否大于等于m，如果是的话，我们就更新结果ans的值，然后找到不满足或者全部分解完为止，每找到一个就将末尾的元素删除，就能算出中位数。

```
#define ll long long
ll ksm(ll a,ll b) {
        ll ans = 1;
        for(;b;b>>=1LL) {          //把int类型的变量1转化为long long，然后再赋值给long long类型的变

            if(b & 1) ans = ans * a % mod;     //如果是奇数

            a = a * a % mod;
        }
        return ans;
}
ll lowbit(ll x){return -x & x;}
const int N = 2e6+10;
int n,m,q,a[N];
```

中位数切分

解法

排序完后，每次找到中间位置，看是否大于等于m，如果是的话，我们就更新结果ans的值，然后找到不满足或者全部分解完为止，每找到一个就将末尾的元素删除，就能算出中位数。

```
int main()
{
        int t;
        scanf("%d",&t);
        while(t--){
                scanf("%d%d",&n,&m);
                for(int i = 1;i <= n; ++i) scanf("%d",&a[i]);
                //对输入的数字进行排序;
                sort(a+1,a+n+1);
                int ans = 0;
                int r = n;
                while(r >= 1){
                        if(a[(1+r)>>1] >= m) r--,ans++;      //a[(1+r)>>1]  找到中间位置的数。
                        else break;
                }
                if(ans == 0) puts("-1");
                else printf("%d\n",ans);
        }


        return 0;
}
```

例如： 5 3
5 2 3 3 2
n=5, m = 3时
r = 5，a[(1+r)>>1] = a[3]=3,
ans++;

中位数切分 性能分析

分析

1、&可以快速地判断一个整数n的奇偶性
if（n&1)==1 则n是奇数 else 则n是偶数（x表示未知）
当n是奇数时 n转换成二进制为xxxxxxxxxxxx1
n是偶数数 n转换成二进制为 xxxxxxxxxxxx0
n转换成二进制为000000000001

2、C语言中的 >>= 意思为：右移后赋值
代码示例为：

```
x = 8;
x >>= 3;
```

```
x = 8;    // 00000000 00000000 00000000 00001000 十进制8的二级制数值
x >>= 3; //  00000000 00000000 00000000 00000001 右移3位赋值给x，此时x的十进制数值为1
```

算法时间代价：
每次找到中间位置，看是否大于等于m，如果是的话，我们就更新结果ans的值，每找到一个就将末尾的元素删除，然后找到不满足或者全部分解完为止。
时间复杂度为：O(n)

两个有序序列的中位数

问题描述

已知有两个等长的序列S1, S2, 设计函数求S1与S2**并集的中位数**。
有序序列A0,A1,… ,AN−1的中位数指A(N−1)/2的值,即[(N+1)/2]个数（A0为第1个数）。

输入格式:
输入分三行。第一行给出序列的公共长度N（0<N≤100000），随后每行输入一个序列的信息，即N个非降序排列的整数。数字用空格间隔。

输出格式:
在一行中输出两个输入序列的并集序列的中位数。

```
5
1 3 5 7 9
2 3 4 5 6
```

```
4
```

两个有序序列的中位数

解法一

算法思想：只使用一个数组，将输入的两行整数都存到一个数组里，再进行排序，最后输出数组的中位数即可。

```
int main(){
        int n,i;
        int str[200000];
        scanf("%d",&n);
        for(i=0;i<n;i++){
                scanf("%d",&str[i]);
        }
        for(i=n;i<2*n;i++){
                scanf("%d",&str[i]);
        }
        sort(str,2*n);

        int a=(2*n-1)/2;   //找到中间位置下标
        printf("%d\n",str[a]);
        }
```

两个有序序列的中位数

解法一

算法思想：只使用一个数组，将输入的两行整数都存到一个数组里，再进行排序，最后输出数组的中位数即可。

```
void sort(int data[], int n) {    //排序算法
    int i,j,mix,temp;
    for(i=0;i<n-1;i++) {
        int mix = i;
        for(j=i+1;j<n;j++) {
            if(data[j] < data[mix]) {
                mix = j;
            }
        }
        if(i != mix) {
            temp = data[i];
            data[i] = data[mix];
            data[mix] = temp;
        }
    }
}
```

```
void Quick_Sort(int *arr,int n){
    int begin=0;
    int end=n-1;
    if(begin > end)
        return;

    int tmp = arr[begin];
    int i = begin;
    int j = end;
    while(i != j){
        while(arr[j] >= tmp && j > i)
            j--;
        while(arr[i] <= tmp && j > i)
            i++;
        if(j > i){
            int t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }
    arr[begin] = arr[i];
    arr[i] = tmp;
    Quick_Sort(arr, begin, i-1);
    Quick_Sort(arr, i+1, end);
}
```

同样可以用快速排序

两个有序序列的中位数

解法一

本题关键在于题目指出输入为整数位非降序排列，就是有可能为升序，有可能为等序。所以要进行排序。

算法时间复杂度为：主要由sort()函数决定，平均时间复杂度为 n(n-1)/2， 则总时间复杂度即为O(n^2);
如果用快速排序，快速排序的最差时间复杂度和冒泡排序是一样的都是O(n^2)，平均时间复杂度为O（n log n）

两个有序序列的中位数

解法二

算法思想：循环比较两个数组的中位数的大小，直到两个数组的起始下标等于结束下标时返回两个数中小的那个数，就是整个数组的中位数。

```
int Median_Search(int a[] , int b[] , int n){
    int first_a = 0 , first_b = 0 , end_a = n-1 , end_b = n-1;
    int mid_a , mid_b ;
    while( first_a != end_a || first_b != end_b ){
        mid_a = (first_a+end_a)/2 ;
        mid_b = (first_b+end_b)/2 ;
        if(a[mid_a]==b[mid_b])
            return a[mid_a];
        if(a[mid_a]<b[mid_b]){
            if( (first_a+end_a)%2 == 0 ){
                first_a = mid_a ;
                end_b = mid_b ;
            }
            else{
                first_a = mid_a + 1 ;
                end_b = mid_b ;
            }
        }
        else{
            if( (first_a+end_a)%2 == 0 ){
                end_a = mid_a ;
                first_b = mid_b ;
            }
            else{
                end_a = mid_a ;
                first_b = mid_b +1 ;
            }
        }
    }
    return
a[first_a]<b[first_b]?a[first_a]:b[first_b];
}
```

两个有序序列的中位数

解法二

算法时间复杂度：输入为O（n）。while循环为O（n）。平均时间复杂度为O（n）算法时间复杂度为O（n）。

```
5
1 3 5 7 9
2 3 4 5 6
4
--------------------------------
Process exited after 6.206 seconds with return value 0
请按任意键继续. . .
```

```
int main()
{
    int n,a[50],b[50];
    scanf("%d",&n);
    for(int i = 0 ; i < n ; i ++){
        scanf("%d",&a[i]);
    }
    for(int i = 0 ; i < n ; i ++){
        scanf("%d",&b[i]);
    }

printf("%d",Median_Search(a,
b,n));
    return 0;
}
```

两个有序序列的中位数

解法三

算法思想：1、从下标为0开始比较a、b两个数组的大小。
2、如果a大，b的下标向后移一位。
3、如果b大，a的下标向后移一位。
4、直到a,b的下标相加等于n-1时，退出循环。
5、比a、b此时下标小的数和比a、b下标大的数相等，/6、a、b此时的下标对应的两个数中小的那个数一定是中位数。

```
int main(){
        int n, i, j = 0, k = 0;
        scanf("%d", &n);
        int a[n], b[n];
        for(i = 0; i < n; i++)
                scanf("%d", &a[i]);
        for(i = 0; i < n; i++)
                scanf("%d", &b[i]);
        ShellSort(a,n);
        ShellSort(b,n);
        while(j + k < n - 1){
                if(a[j] >= b[k])
                        k++;
                else
                        j++;
        }

        printf("%d", a[j]>b[k]?b[k]:a[j]);
        return 0;
}
```

如果两组序列无需排序，则此法又叫分治法。

两个有序序列的中位数

解法三

```
//希尔排序
void ShellSort(int  A[], int n)
{
        int gap = n;
        while (gap>1)
        {
                //每次对gap折半操作
                gap = gap / 2;
                //单趟排序
                for (int i = 0; i < n - gap; ++i)
                {
                        int end = i;
                        int cur= A[end + gap];
                        while (end >= 0)
                        {
                                if (cur< A[end])
                                {
                                        A[end + gap] = A[end];
                                        end -= gap;
                                }
                                else
                                {
                                        break;
                                }
                        }
                        A[end + gap] = cur;
                }
        }
}
```

1、先选定一个小于N的整数gap作为第一增量，然后将所有距离为gap的元素分在同一组，并对每一组的元素进行直接插入排序。
2、然后再取一个比第一增量小的整数作为第二增量，比如gap/2；重复上述操作…
3、当增量的大小减到1时，就相当于整个序列被分到一组，进行一次直接插入排序，排序完成。

两个有序序列的中位数

解法三

算法时间复杂度为：主要由sort(a,n)函数决定，平均时间复杂度为n， 则总时间复杂度即为O(n^2);
同样，如果用ShellSort(a,n)，则时间复杂度为nlogn;

排序算法

时间复杂度表

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	In-place	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(1)$	In-place	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Out-place	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	In-place	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Out-place	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Out-place	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Out-place	稳定

两个有序序列的中位数

分治法

算法思想介绍：

假定A[0,……,n-1] 和 B[0,……,n-1] 是输入数组，
令k = [n/2]（向下取整），取A[k] 和 B[k]比较。
如果A[k] == B[k]，则A[k]就是所求的中位数，
直接输出，算法结束；

如果A[k] < B[k]，则A[0,……,k-1]
和 B[k+1,……,n-1] 不含有C的中位数，直接舍弃，
下一步递归查找的子问题是A[k,……,n-1]
和 B[0,……,k]（注：C中位数必定在这个区间之
内）；

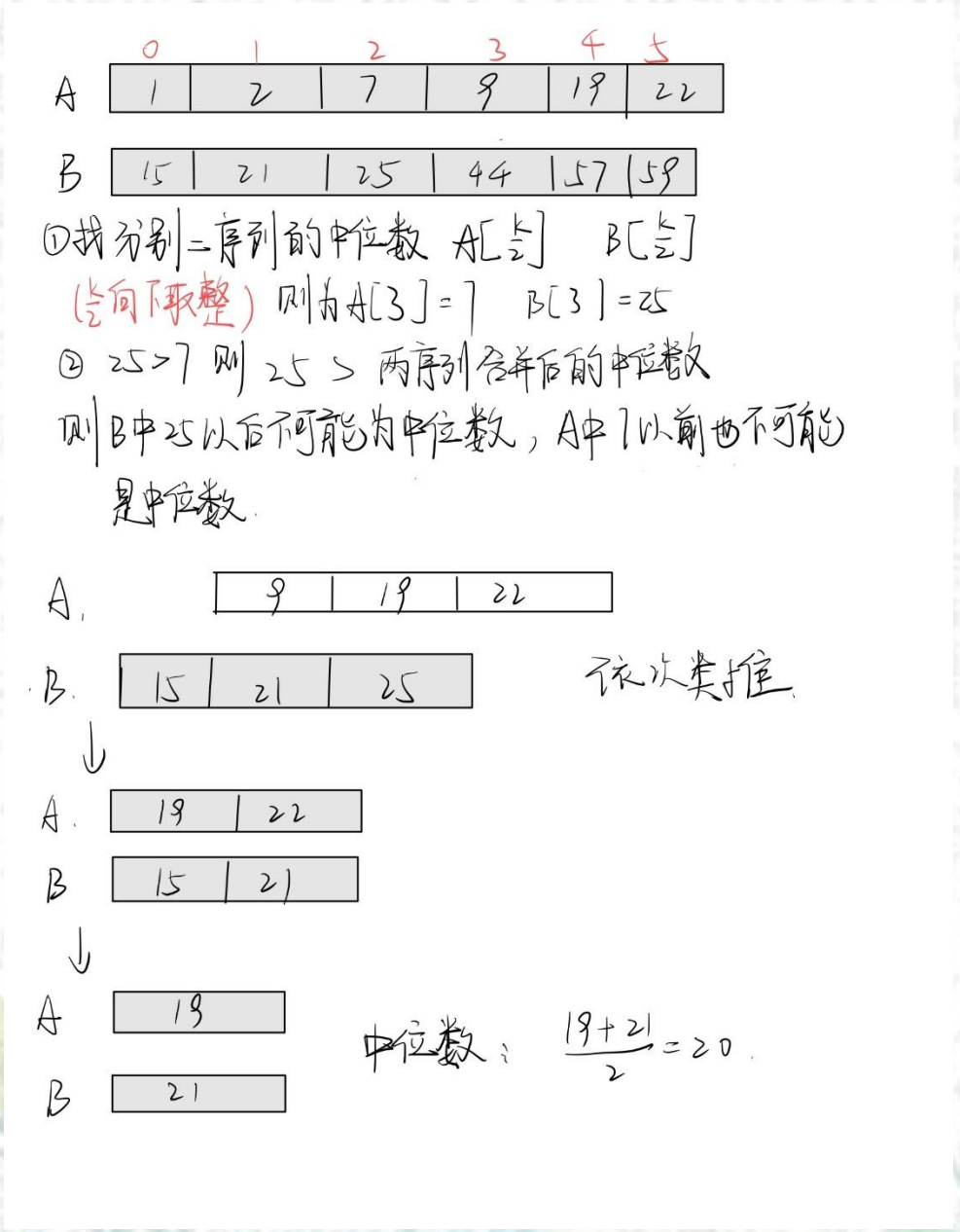
如果A[k] > B[k]，则A[k+1,……n-1]
和 B[0,……,k-1]不含有C的中位数，直接舍弃，下
一步递归查找的子问题是A[0,……,k]
和 B[k,……,n-1].

当n == 1，A和B中都只剩下一个数，那么取中
位数mid = (A[1] + B[1])/2，算法结束；

两个有序序列的中位数

分治法

一个分治法求解的实例
其解题过程



两个有序序列的中位数

分治法

算法求解步骤

```
float midnum(float a[], int s1, int t1, float b[], int s2, int t2)
{
    int m1, m2;
    if (s1 == t1 && s2 == t2)
        return (a[s1] + b[s2])*0.5;
    else
    {
        m1 = (s1 + t1) / 2;//序列1的中位数
        m2 = (s2 + t2) / 2;//序列2的中位数
        if (a[m1] == b[m2])
            return a[m1];//若两个中位数相等则返回该中位数
        else if (a[m1] < b[m2])
        {
            backpart(s1, t1);//序列1取后半部分
            prepart(s2, t2);//序列2取前半部分
            return midnum(a, s1, t1, b, s2, t2);
        }
        else
        {
            backpart(s2, t2);//序列2取后半部分
            prepart(s1, t1);//序列1取前半部分
            return midnum(a, s1, t1, b, s2, t2);
        }
    }
}
```

```
void prepart(int& s, int& t)
{
    int m = (s + t) / 2;
    t = m;
}
```

```
void backpart(int& s, int& t)
{
    int m = (s + t) / 2;
    //序列中有奇数个元素
    if ((s + t) % 2 == 0)
        s = m;
    //序列中有偶数个元素
    else
        s = m + 1;
}
```

两个有序序列的中位数

分治法

时间复杂度：
该问题每次将子问题减半处理，时间复杂度为O（logn）。

参考资料

知乎：https://zhuanlan.zhihu.com/p/102182602；
CSDN：
https://blog.csdn.net/m0_46201544/article/details/122722787?ops_request_misc=&request_id=&biz_id=102&utm_term=%E4%B8%AD%E4%BD%8D%E6%95%B0%E5%88%87%E5%88%86&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduweb~default-0-122722787.142^v9^control,157^v4^control&spm=1018.2226.3001.4187




THANKS!